

Private Information Retrieval for Bitcoin Light Clients

Weikeng Chen

May 2026

1 Introduction

We are interested in using Private Information Retrieval (PIR) to help Bitcoin light clients synchronize with the Bitcoin network privately. A light client that knows its own keys needs to discover which unspent transaction outputs (UTXOs) it can spend, without revealing its addresses to the servers it queries. PIR enables this by allowing the client to retrieve specific database entries without the server learning which entries were accessed.

1.1 Private Information Retrieval

Private Information Retrieval (PIR) allows a client to retrieve an entry from a database held by one or more servers without revealing which entry is being accessed. Formally, consider a database D of N entries, each of B bits. The client holds an index $i \in [N]$ and wishes to retrieve $D[i]$.

The trivial solution is for the client to download the entire database, achieving perfect privacy at $O(NB)$ communication cost. The goal of PIR is to achieve sublinear communication while preserving privacy.

Single-server (computational) PIR. A single server holds D . The client’s query must be computationally indistinguishable from a query for any other index under some hardness assumption. The server performs computation over the query and the database to produce a response, from which the client can extract $D[i]$. The server’s computation is at least $\Omega(NB)$ in the worst case (otherwise the server trivially learns that certain entries were not queried), though preprocessing and client-side hints can reduce the *online* computation.

Multi-server (information-theoretic) PIR. The database is replicated across $t \geq 2$ non-colluding servers. Privacy holds information-theoretically: no single server (or coalition below the threshold) gains any information about i . The simplest 2-server scheme uses additive secret sharing of a selection vector, achieving $O(N)$ communication per server—already non-trivial compared to $O(NB)$ when B is large.

1.2 PIR Protocol Landscape

We survey practical PIR constructions across two axes: the number of servers and the amount of client-side state. We consider four settings:

- **Single-server:**
 - Almost no client storage: YPIR-over-SimplePIR [MW24], or FHE-based schemes such as OnionPIR [MCR21] and SealPIR [ACLS18]
 - Sublinear client storage without a linear pass: SimplePIR or DoublePIR [HHCG⁺23]
 - Sublinear client storage after a linear pass: Plinko [HPPY25], RMS24 [RMS24], or Piano [ZPZS24]
- **2-server:**
 - Almost no client storage: DPF-PIR [GI14, BGI15, BGI16]

- Sublinear client storage with outsourced hints: Plinko, RMS24, or Piano with hint generation delegated to a second server; or stateful PIR via pseudorandom permutations [AR26a, KCG21]

1.2.1 Single-Server PIR

We focus on single-server PIR without per-client storage on the server and allow server to preprocess the database. For protocols that do not require client storage (or if the client storage is just for some data-independent parameters), YPIR-over-SimplePIR has achieved best performance and throughput for larger blocks, in comparison to Hintless PIR [LMRS24], Tiptoe [HDCGZ23], and Spiral [MW22].

If we allow the client to make data-dependent but sublinear storage for “hints”, then we get better results with SimplePIR or DoublePIR, where the client downloads the hints before making any query. Note that DoublePIR is most suitable for single-bit or few-bit entries, and does not perform well when the entry size exceeds several tens of bits. This is because: (1) while the client hint is independent of the database size, it scales linearly with the entry size; and (2) this linear scaling has a very large multiplicative constant factor. This makes DoublePIR suitable for certain PIR use cases, but not the other. When using SimplePIR or DoublePIR, the server time appears to improve around 2 times to an order of magnitudes with the sublinear hints over those that don’t require hints.

If we do one step further and allow the client to download and make a linear pass over the database to generate sublinear hints (the client downloads and performs the linear pass in the streaming manner with only sublinear storage needed), then we can get much better performance with Plinko, RMS24, or Piano. The server time can be reduced by potentially two orders of magnitude. This setting works well for many devices with access to home internet or WiFi network where the network is not metered, but not all.

1.2.2 Two-Server PIR

We now consider PIR that assumes two servers that are non-colluding and non-communicating, meaning that the two servers do not need to know each other, and it is only the user who engages with both servers. In fact, any two trustworthy servers that have the database can be selected by the user to run this protocol. This setting is especially relevant to Bitcoin peer-to-peer (P2P) network as a user can discover two full nodes that provide this capability and use their help. The caveat is that the user needs to trust that these two servers do not collude with each other. If not, the two colluding servers can immediately learn the user’s query.

The easiest way to achieve 2-server PIR is to use DPF-PIR, which is based on distributed point functions (DPF) [GI14, BGI15, BGI16]. Given a share of DPF keys of size $O(\log N)$, each server can expand it to N bits that look random, where the two servers’ bits differ exactly on the location that the user queries. Each server retrieves the entries with bit 1 and computes an XOR sum of them. The user finally performs an XOR over the two XOR sums from the two servers and obtains the desired query result. It requires $O(NB)$ server compute time with B being the entry size, but the computation is extremely lightweight. Usually, the compute can answer a PIR query close to the time that it takes to do a linear pass of the database in the storage medium (which usually is the main memory).

TEE-hardened deployment. A Trusted Execution Environment (TEE) such as AMD SEV-SNP provides hardware-enforced memory encryption and remote attestation: the TEE’s runtime state is opaque to the host operating system and hypervisor, and a remote party can cryptographically verify the exact binary executing inside the TEE. In a DPF-PIR deployment, running one of the two servers inside a TEE removes that server’s operator from the trust base entirely—even a malicious operator cannot read the DPF key share or XOR-sum response from TEE-protected memory. The client attests the TEE before issuing queries, verifying that it runs the expected PIR server binary. This refines the 2-server trust assumption from “two independent operators” to “one trusted operator + one honest piece of silicon” (plus trust in the TEE vendor’s root of trust). Concretely, a Bitcoin light client can rely on a single community-run full node for the first DPF share while outsourcing the second share to any cloud provider offering confidential VMs, without needing to trust that provider’s personnel.

If we allow hint generation, as is the case in the single-server setting, we know that the server can answer the PIR

queries in sublinear time, which is extremely efficient that seems to “surpass the memory speed.” Previously in the single-server setting, a significant downside is that the client needs to download the entire database (in a streaming manner) to generate the hint locally. Now, in the 2-server setting, the user can outsource the hint generation to one of the servers (“hint server”), and the hint can be used to query any server other than the hint server. This avoids the need for the user to download the database, and the user can directly download the hint, though it still requires sublinear client storage.

1.2.3 Query Volume Considerations

When we compare these different solutions, it is important to understand how many queries a user would make. For example, if a user only makes one query, then all the compute and communication costs for the hint generation may not be worthwhile if it is single-use. The overhead is not just about the client, but also the server. For single-server protocols that require a linear pass, a server needs to serve the entire database to this user. For 2-server protocols with outsourced hint generation, the hint server bears the computation cost for doing that linear pass over its local copy of the database.

This is especially relevant to Bitcoin light client synchronization because a light client may maintain a small wallet that only cares about UTXOs that the wallet can spend. In this situation, the light client knows exactly how many queries it needs to make, and after those queries, the light client has no need to access this database anymore as it has been synced up to the state that is current to the database. In this case, the client may only make one or a few queries to the database, and therefore the overhead for hint generation may not be worthwhile. An example of such a use case is when a light client imports a seed phrase or a private key, in which case it needs to identify all the UTXOs associated with the imported wallets, but this is a one-time thing.

Despite this concern, PIR is still superior than asking the light client to download the compact block filters for all the blocks since the imported key is first used (if known) and fetch a number of blocks. First, the compact block filters for the entire Bitcoin history can still be a few gigabytes. Second, the usual way to fetch relevant blocks is to request these blocks directly from a full node by giving the block height, which leaks a little bit of privacy (which blocks this new wallet has a UTXO for). Whether this privacy leakage is acceptable depends on the situation and the user. Our goal building the PIR solution is to help those users who want to avoid this privacy leakage (which, as we will show, it is easy to avoid).

1.2.4 Integrity and Verifiability

Modern PIR naturally achieves privacy even against malicious servers, so we only need to focus on integrity. There are many ways to achieve integrity, some of which offer integrity against selective failure attacks [Kil88, NP99, DNRS99, CNS07, LP11, KS06], while some do not. Whether it is okay to ignore selective failure attacks depends on the user and the use case. For example, a user who receives a potentially selectively failed response can continue to finish the rest of the protocol using random queries, pretending to have received a valid response, and then fetch the data from other servers.

If we ignore selective failure attacks, the most practical solution for single-server and 2-server solutions appears to be KZG polynomial commitment [KZG10], where the opening proofs are generated using the Feist-Khovratovich batch technique [FK23]. An important limitation is that it uses elliptic curves, and therefore is not post-quantum. The KZG opening proof would add about 16 bytes to each entry. To reduce the PIR compute overhead caused by the additional opening proof, we can merge a few entries into one larger entry, and still only one opening proof is needed for the larger entry. With the same setup parameters of KZG, all the nodes in the network should have the same KZG polynomial commitment.

This solution, however, fails to address selective failure attacks. Selective failure attacks refer to the case when a malicious server manipulates a subset S of the database entries such that if the user is reading an entry in S , it would fail, and if the user is reading an entry outside of S , it would succeed. The concern is that such a selective failure, which may be realized by the malicious server through a change of the user’s subsequent actions, can leak if

the user is reading an entry in S .

To address selective failure attacks, we can choose protocols with this additional verifiability guarantee against selective failure attacks. For single-server protocols without a linear pass, RLP26 [RLP26] is a good candidate, which improves over VeriSimplePIR [dL24] and significantly reduces the client hint using some one-time computation.

- **Single-server:**

- Almost no client storage: RLP26 [RLP26]
- Sublinear client storage after a linear pass: Plinko, RMS24, or Piano with Merkle proofs, or verifiable BALANCED-PIR [AR26b]

- **2-server:** Plinko, RMS24, or Piano with Merkle proofs, or verifiable BALANCED-PIR with outsourced hint generation

Our own implementation takes a more direct route: we build SHA-256 Merkle trees over the cuckoo bins of the live database and verify retrieved bins by issuing the proof queries through the same PIR primitive the client already uses for data. This keeps the entire system post-quantum, avoids a trusted setup, and lets the same verification layer serve both the DPF and HarmonyPIR backends without protocol changes. We describe the construction in full in Section 7.

1.3 Our Contributions

We implement three distinct PIR protocols for Bitcoin UTXO lookups, each offering different trade-offs between trust assumptions, communication cost, and server computation:

- **DPF-PIR** (Section 4): A 2-server stateless protocol based on Distributed Point Functions. Requires no client state and minimal communication, but assumes two non-colluding servers.
- **HarmonyPIR** (Section 5): A 2-server stateful protocol [AR26a] that trades a one-time offline hint download for dramatically cheaper online queries. The hint server and query server can be different nodes.
- **OnionPIR** (Section 6): A single-server protocol based on fully homomorphic encryption [MCR21, CR25]. Eliminates the non-colluding assumption entirely at the cost of higher computation and communication.

All three protocols share the same data representation (Section 2) and batch PIR framework (Section 3), enabling a unified system where clients can choose the protocol best suited to their threat model and network conditions.

We complement the three backends with two cross-cutting features. First, an integrity layer (Section 7) that builds per-group SHA-256 Merkle trees over cuckoo bins and verifies retrieved data by running the proof queries through the same PIR primitive, preserving post-quantum security and avoiding trusted setup. Second, support for *delta databases* (Section 8) that encode the diff between successive UTXO snapshots, letting a returning client incrementally sync by retrieving a small delta instead of re-downloading an entire snapshot.

We also provide a practical deployment (Section 11) including browser-based clients via WebAssembly [W3C19] and an Electrum wallet plugin [Voe11] with all three backends, a multi-database server architecture, and a client-side sync planner that chooses between full and delta databases.

1.4 Paper Organization

Section 2 describes the Bitcoin UTXO model, our data representation, and techniques for reducing the UTXO set size. Section 3 presents the PIR-variant-agnostic batch PIR framework using Probabilistic Batch Codes. Sections 4, 5, and 6 detail the three protocol instantiations: DPF-PIR, HarmonyPIR, and OnionPIR, respectively. Section 7 describes the per-group Merkle integrity layer, and Section 8 describes delta databases and incremental sync. Section 10 compares the three protocols across multiple dimensions. Section 11 describes the implementation and deployment, including the multi-database catalog, browser and wallet integration.

2 Data Representation

2.1 Bitcoin UTXO Model

Bitcoin uses an Unspent Transaction Output (UTXO) model. Each transaction consumes previous outputs (inputs) and creates new outputs. The set of all outputs that have not yet been spent forms the *UTXO set*, which represents the current state of all spendable coins.

Each UTXO is uniquely identified by a pair (TXID, vout), where TXID is the 32-byte hash of the creating transaction and vout is the output index within that transaction. Each UTXO carries an *amount* (in satoshis) and a *ScriptPubKey*—a locking script that specifies the conditions under which the output can be spent.

Common ScriptPubKey types include:

- **P2PKH** (Pay-to-Public-Key-Hash): Legacy addresses starting with 1
- **P2SH** (Pay-to-Script-Hash): Script hash addresses starting with 3
- **P2WPKH / P2WSH** (Pay-to-Witness-*): SegWit addresses starting with bc1q
- **P2TR** (Pay-to-Taproot): Taproot addresses starting with bc1p

A *light client* knows its own private keys (and thus its addresses and ScriptPubKeys) but does not maintain a full copy of the blockchain. To spend its coins, the light client needs to discover which UTXOs are locked to its ScriptPubKeys and retrieve the corresponding (TXID, vout, amount) tuples. PIR enables this discovery without revealing the client's addresses to the server.

2.2 Data Organization

Sort and group. The server sorts UTXOs by ScriptPubKey hash, then packs them into fixed-size chunks. Each chunk stores complete UTXO records including full 32-byte TXIDs, vout indices, and amounts. Multiple ScriptPubKeys may share a chunk via bin-packing.

Index map. Maps each ScriptPubKey (via its RIPEMD-160 hash) to the chunk location and data length. This is organized as a cuckoo hash table for $O(1)$ keyword-to-index lookup [PR01]. Each index slot is 13 bytes: an 8-byte fingerprint tag (derived from the script hash, used to select the correct slot within a retrieved bin), a 4-byte starting chunk identifier, and a 1-byte chunk count.

Chunk map. Stores the actual UTXO data in fixed-size 40-byte blocks. Full 32-byte TXIDs are stored directly in the chunks, eliminating a separate TXID resolution step. Each chunk slot is 44 bytes: a 4-byte chunk identifier followed by 40 bytes of UTXO data. The chunk map is also organized as a cuckoo hash table for PIR-compatible retrieval.

2.3 Cuckoo Hashing for Keyword Lookup

Cuckoo hashing [PR01] is a hashing scheme that provides worst-case $O(1)$ lookup time. To store n items, we maintain a table of T bins, each holding s slots, with h independent hash functions $H_1, \dots, H_h$.

Each item x can reside in any of its h candidate bins $H_1(x), \dots, H_h(x)$. To insert x : if any candidate bin has a free slot, place x there. Otherwise, evict a random existing item from one candidate bin, insert x , and re-insert the evicted item recursively. With appropriate choices of h and s (e.g., $h=2, s=3$ or $h=3, s=2$), load factors above 95% are achievable.

PIR by keyword. Cuckoo hashing enables PIR by keyword (as opposed to PIR by index). Given a keyword w , the client computes its h candidate bin indices and issues PIR queries for those bins. Upon retrieving the bin contents, the client checks which slot (if any) contains an entry matching w . This transforms the problem of searching an unordered key-value store into a constant number of PIR-by-index queries.

We use two distinct terms throughout the rest of the paper to avoid collisions. A *bin* is a location in one of these

keyword-lookup cuckoo tables: a row of s slots that a PIR query retrieves atomically. A *group* refers to the outer sharding used by batch PIR (Section 3): the database is partitioned into K groups, one cuckoo table per group, and a batch of client queries is distributed across the K groups so that each group answers at most one real query per round.

2.4 Reducing the UTXO Set

Since the PIR service is designed for light clients who do not frequently run the Bitcoin node for synchronization or for newly imported wallet addresses from a regular user, we can realistically expect that the user only makes a few transactions, keeps only a few UTXOs in the chain, and has no intention to create large number of dust outputs (a typical pattern of some Bitcoin layer-2 protocols). Therefore, taking this in mind, we can reduce the UTXO set by filtering out scriptPubKeys that show different user patterns, or, if not filtering out the entire scriptPubKey, we can selectively filter out certain transactions or certain UTXOs that match some profiles.

For example, a scriptPubKey that has been involved in a significantly above average number of UTXOs is likely not a regular user and probably should run its own full node to have the full security and privacy.

Another example is that we can filter out UTXOs that are of dust amount. It is possible that a honest user has dust UTXOs and may want to use them in the future, but given that each dust UTXO is worth probably like 0.4 USD, we can say that they are of a lower priority, and a user who sincerely wants to use them should first try to avoid generating dust UTXOs as it should not occur naturally, and if the user sincerely wants to recover from the dust amount, the user is recommended to use an RPC server (maybe through Tor), or if strict privacy is a concern, run a full node—honestly speaking, if the user wants to consolidate these dust UTXOs, the other full nodes are going to learn it sooner or later anyway because they can wait for the consolidation transaction.

If we worry that the dust amount filtering might be excessive, we can also target certain protocols that are known to generate a large number of dust UTXOs, such as the Omnilayer, by looking into the OP_RETURN output next to the dust UTXO and checking if it starts with “omni”.

We can also make a list of known exchange wallet addresses and filter them out given that these crypto exchange is definitely not running a light client. There are some services for this purpose, such as Arkham, that provides a list of identified addresses of notable market participants (who are unlikely using this light client).

3 Batch PIR

A light client typically needs to look up multiple ScriptPubKeys simultaneously. This section describes how we use Probabilistic Batch Codes (PBC) to amortize the cost of multiple PIR queries, and how the two-level architecture (index map then chunk map) composes with batching. The batch PIR framework described here is PIR-variant-agnostic: it works with any underlying single-query PIR scheme, including the 2-server DPF-PIR instantiation in Section 4.

3.1 Batch PIR and Probabilistic Batch Codes

Batch PIR [IKOS04] addresses the problem of retrieving m entries simultaneously. The naive approach—issuing m independent PIR queries—costs m times the single-query cost. Batch PIR aims to reduce this by amortizing work across queries.

Probabilistic Batch Codes (PBC) achieve batch PIR by distributing m queries across K groups (with $K > m$) such that each group contains at most one real query. The construction uses cuckoo hashing:

1. For each of the m queries, compute h candidate groups using cuckoo hash functions.
2. Place queries into groups via cuckoo insertion with eviction chains.
3. Pad all empty groups with random dummy queries.
4. Issue K independent PIR queries in parallel—one per group.

Since each group holds at most one real query, the server processes K single-entry PIR queries. For DPF-PIR, this means generating K DPF key pairs and having each server perform K XOR-sum computations. The effective cost per query drops from $O(NB)$ to $O(NB \cdot K/m)$ when the server can share work across the K evaluations.

Each of the K groups is itself a cuckoo hash table in the sense of Section 2, where the inner rows are called *bins*. We consistently use “group” for the outer PBC sharding and “bin” for the inner cuckoo row; this matches the terminology used throughout our codebase.

3.2 Two-Level Query Architecture

Our system uses a two-level architecture: an *index map* that locates data, and a *chunk map* that stores the actual UTXO data (see Section 2 for the data layout). A complete UTXO lookup requires two rounds of batch PIR:

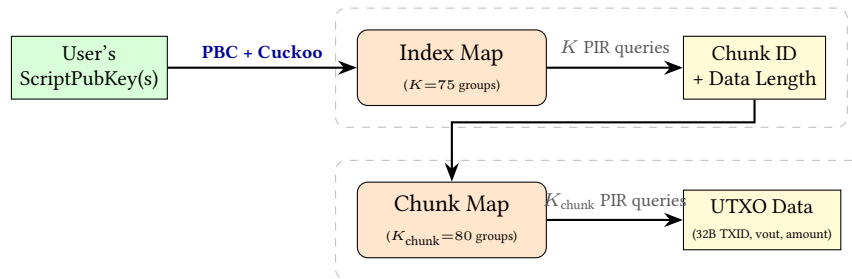


Figure 1: Two-level batch PIR architecture. The client issues K parallel PIR queries per level, using Probabilistic Batch Codes to pack multiple lookups into a single round.

Level 1: Index lookup. The client computes RIPEMD-160 hashes of all queried ScriptPubKeys, derives h candidate groups per query via cuckoo hash functions, and places them into K PBC groups. For each group, the client generates a PIR query targeting the assigned bin in the group’s index cuckoo table and sends it to the server(s). The responses are combined to recover index slots containing starting chunk ids and chunk counts.

Level 2: Chunk retrieval. Using the chunk locations from Level 1, the client repeats the process with K_{chunk} PBC groups over the chunk database. The recovered chunks contain full UTXO records: 32-byte TXIDs, vout indices, and amounts in satoshis—everything needed to construct a spending transaction. Each chunk is 40 bytes; each slot in the chunk cuckoo table is 44 bytes (4-byte chunk identifier plus 40-byte payload).

Distributional PIR. We can apply distributional PIR [LHCG25] to optimize for the common case: most light clients hold few UTXOs under common script types, while whale addresses with thousands of UTXOs likely run full nodes. By tuning the PIR parameters for the more likely queries, the average server cost decreases without eliminating the ability to query less common entries.

Database	Slot Size	Groups (K)	Cuckoo	DPF queries/group	PIR domain
Index (Level 1)	13 B	75	2-hash, 4 slots/bin	2	2^{20}
Chunks (Level 2)	44 B	80	2-hash, 3 slots/bin	2	2^{21}

Table 1: Batch PIR parameters. Each level uses K parallel PIR queries over cuckoo hash tables, one table per group. Each DPF query retrieves one cuckoo bin: $4 \text{ slots} \times 13 \text{ B} = 52 \text{ B}$ for the index level, and $3 \text{ slots} \times 44 \text{ B} = 132 \text{ B}$ for the chunk level.

4 2-Server Stateless PIR via DPF

This section describes the 2-server stateless PIR instantiation of our system, based on Distributed Point Functions (DPF). The client requires no persistent state or hints—only the DPF keys generated fresh for each query session. Combined with the batch PIR framework from Section 3, this yields a complete system for private UTXO lookups.

4.1 Distributed Point Functions (DPF)

A Distributed Point Function (DPF) [GI14, BGI15, BGI16] is a cryptographic primitive that enables efficient 2-server PIR. A *point function* $f_{\alpha,\beta}$ evaluates to β at input α and to 0 everywhere else. A DPF splits this point function into two keys (k_0, k_1) such that:

- **Correctness:** $\text{Eval}(k_0, x) \oplus \text{Eval}(k_1, x) = f_{\alpha,\beta}(x)$ for all x .
- **Security:** Each individual key k_b is computationally indistinguishable from a key for any other point function.

Each key has size $O(\lambda \log N)$ where λ is the security parameter and N is the domain size. To perform PIR on a database of N entries indexed by $[N]$:

1. The client generates DPF keys (k_0, k_1) for the point function $f_{i,1}$ where i is the target index.
2. Server b expands k_b to obtain a vector of N field elements and computes $r_b = \bigoplus_{x=0}^{N-1} \text{Eval}(k_b, x) \cdot D[x]$.
3. The client recovers $D[i] = r_0 \oplus r_1$.

The server computation is $O(NB)$ but consists of simple XOR operations over the database, making it extremely fast in practice—often close to the speed of a single linear pass over the data in memory.

4.2 Implementation

We have implemented a complete system based on the 2-server DPF-PIR approach with Probabilistic Batch Codes (PBC), as described in Section 3. The implementation consists of database generation tools (Rust), two PIR servers (Rust with `tokio`), and client libraries (Rust CLI and TypeScript/Web). The two-level architecture from Figure 1 eliminates the TXID resolution step of earlier designs by storing full 32-byte TXIDs directly in the UTXO chunks.

4.2.1 System Architecture

Each server maintains identical copies of the index and chunk databases, loaded via memory-mapped files (`mmap2`). The client connects to both servers over WebSocket and distributes DPF key shares, so neither server learns the query target unless they collude. Communication uses a compact binary protocol supporting database discovery, batch PIR queries, and connection health checks.

4.2.2 Database Generation

The offline pipeline transforms raw Bitcoin blockchain data into two PIR-queryable databases. Each stage is a separate Rust binary in the `build` crate.

Pipeline overview:

1. **UTXO extraction:** Parse the UTXO snapshot (`dumptxoutset`) to extract ScriptPubKey hashes, full 32-byte TXIDs, vout indices, and amounts.
2. **Chunk generation:** Group UTXOs by ScriptPubKey, serialize with VarInt encoding, and pack into fixed-size 40-byte blocks. Each block stores complete records including full TXIDs.
3. **Index construction:** Build an index mapping each ScriptPubKey hash to its chunk location and count. Each index slot is 13 bytes: an 8-byte fingerprint tag (derived from the script hash) + 4-byte starting chunk id + 1-byte

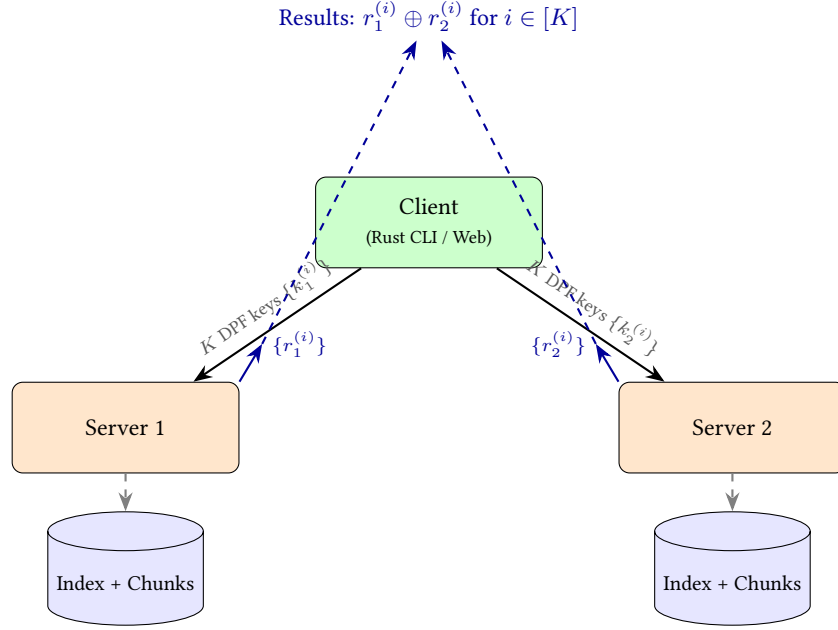


Figure 2: Two-server batch DPF-PIR architecture. The client generates K DPF key pairs (one per PBC group) and sends the shares to each server. Each server computes K XOR sums in parallel; the client combines responses to recover all queried entries.

chunk count.

4. **Cuckoo hashing:** Organize both databases into cuckoo hash tables for PBC-compatible batch retrieval. Both use 2 cuckoo hash functions; the index database has 4 slots per bin and the chunk database has 3 slots per bin, both achieving $>95\%$ load factor with zero stash.
5. **(Optional) Per-group Merkle:** Build per-group SHA-256 Merkle trees over the cuckoo bins, emit flat sibling tables (one file per level), a tree-top cache, and a super-root, enabling integrity verification through the same DPF primitive (Section 7).
6. **Verification:** Validate that all entries are retrievable from the cuckoo tables.

Dual role of cuckoo hashing. Cuckoo hashing serves two purposes in our system. First, it provides PIR-by-keyword: given a ScriptPubKey, the client computes candidate bin indices to locate data without scanning. Second, it implements PBC for batch PIR: when querying m entries, each query maps to 3 candidate groups; the client places queries into K total groups via cuckoo insertion, and all K DPF queries execute in parallel. The index level uses 2 cuckoo hash functions with 4 slots per bin, and the chunk level uses 2 cuckoo hash functions with 3 slots per bin. Each DPF query returns one cuckoo bin: $4 \times 13 = 52$ bytes for the index level, and $3 \times 44 = 132$ bytes for the chunk level.

4.2.3 Query Protocol

A complete UTXO lookup requires two rounds of batch PIR:

Level 1: Index lookup. The client computes RIPEMD-160 hashes of all queried ScriptPubKeys, derives 3 candidate groups per query via cuckoo hash functions (splitmix64-based), and places them into $K=75$ PBC groups. For each group, the client generates 2 DPF key pairs (one per cuckoo hash function in the 2-hash index cuckoo) and sends one share of each to each server. The servers expand all DPF keys and return XOR sums. The client combines responses to recover 13-byte index slots containing fingerprint tags, starting chunk ids, and chunk counts. The client matches the recovered tag against the expected tag for its ScriptPubKey to identify the correct slot within the 4-slot

cuckoo bin.

Level 2: Chunk retrieval. Using the chunk locations from Level 1, the client repeats the process with $K_{\text{chunk}}=80$ PBC groups over the chunk database. The chunk cuckoo uses 2 hash functions with 3 slots per bin, so the client generates 2 DPF key pairs per group. The recovered 132-byte results ($3 \text{ slots} \times 44 \text{ bytes each}$) contain chunk identifiers and full UTXO records: 32-byte TXIDs, vout indices, and amounts in satoshis—everything needed to construct a spending transaction.

4.2.4 Client Implementations

Rust CLI. Command-line tool that takes ScriptPubKeys as hex, performs the two-level batch PIR protocol, and outputs decoded UTXO data.

Web client (TypeScript). Browser-compatible library using WebSocket connections. Implements DPF key generation, cuckoo hashing, and PBC placement in pure TypeScript. Supports Bitcoin address input (P2PKH, P2SH, P2WPKH, P2WSH, P2TR) with automatic conversion to ScriptPubKey.

Electrum plugin (Python). Integration with the Electrum Bitcoin wallet [Voe11] that replaces its native synchronizer with PIR-based UTXO discovery. See Section 11 for details on all client implementations.

4.3 Reducing the Non-Collusion Assumption with TEEs

The security of DPF-PIR rests on the assumption that the two servers do not collude. In a decentralized setting such as Bitcoin’s P2P network, a user can select two independent full nodes and hope that they are operated by unrelated parties. This is a reasonable assumption in many cases but not a provable one, and the consequences of a violation are severe: two colluding servers can simply XOR their DPF evaluations and recover the client’s query.

A Trusted Execution Environment (TEE) [Adv20] provides hardware-enforced guarantees that can eliminate the need to trust one of the two server operators. A TEE is a processor feature (e.g., AMD SEV-SNP, Intel TDX) that creates an encrypted virtual machine whose memory is inaccessible to the host operating system, the hypervisor, and anyone with physical access to the machine. A TEE also supports *remote attestation*: a signed report, rooted in a vendor-provisioned key fused into the processor at manufacturing time, that binds a measurement of the VM’s initial state (firmware, kernel, and userspace binary) to a client-supplied nonce.

4.3.1 Attestation Flow

Before sending any PIR traffic, the client performs a one-round attestation handshake with the TEE-hosted server:

1. The client generates a fresh 32-byte nonce and sends it to the server.
2. The TEE firmware produces a signed attestation report containing a REPORT_DATA field set to $\text{SHA256}(\text{"BPIR-ATTEST-V2"} \parallel \text{nonce} \parallel R^*)$ where R^* is the database super-root (Section 7) and pubkey is a long-lived X25519 key generated inside the TEE at boot.
3. The client recomputes the expected REPORT_DATA from its own nonce, the operator-published binary hash and super-root, and the server’s returned public key, and verifies it against the signed report.

If verification succeeds, the client knows three things: (1) the exact expected PIR server binary is executing inside a genuine AMD SEV-SNP TEE; (2) the server is serving the database committed by the published super-root R^* ; and (3) subsequent DPF shares can be encrypted to the TEE-resident key, guaranteeing they cannot be intercepted by the host.

4.3.2 What the TEE Changes

With one DPF server inside a TEE, the trust model for DPF-PIR improves as follows:

- **Before:** The client trusts that two independent server operators do not collude. Each operator has full access to the DPF key share and XOR-sum response on their machine.
- **After:** The client trusts one independent operator (Server 1) plus the TEE vendor’s root of trust (Server 2). The operator of Server 2 is removed from the trust base—they cannot inspect TEE memory, modify the running binary, or substitute the database without detection. Collusion between Server 1 and Server 2’s operator is harmless because Server 2’s operator has no visibility into the TEE.

The TEE also provides a practical deployment benefit. In our production setup, the TEE-hosted server (Server 2) runs on a commodity cloud VM with AMD SEV-SNP enabled, while the primary server (Server 1) is operated as a community resource. The attestation measurement pins are published alongside each binary release, so any client can verify the TEE independently before connecting.

4.3.3 Scope and Limitations

The TEE does not eliminate all trust assumptions. The client still trusts: the TEE vendor (AMD) and its supply chain; the correctness of the SEV-SNP firmware and hardware implementation; and the operator of Server 1. Side-channel attacks (cache timing, power analysis) against SEV-SNP are an active research area and are not addressed by the attestation mechanism. Finally, the TEE protects only the server side of the protocol; the client’s own machine remains outside any hardware trust boundary.

5 2-Server Stateful PIR via HarmonyPIR

This section describes the 2-server *stateful* PIR instantiation based on HarmonyPIR [AR26a], which uses pseudo-random permutations (PRPs) to organize hints that enable sublinear online queries. Unlike DPF-PIR, HarmonyPIR requires a one-time offline phase where the client downloads precomputed hints, but in return achieves dramatically lower per-query communication and server computation.

5.1 Overview

HarmonyPIR is a stateful PIR protocol in the offline/online model, following the line of work initiated by Checklist [KCG21]. The system involves two servers with distinct roles:

- **Hint server** (offline): Computes and streams PRP-based hint parities to the client. This is a one-time operation per session.
- **Query server** (online): Answers indexed lookups by returning the XOR of requested database entries. Each query requires only $O(\sqrt{N})$ server work.

The key trade-off is: HarmonyPIR incurs a large offline download (hundreds of megabytes of hints) but achieves per-query online communication and server computation that is *sublinear* in the database size. This makes it well-suited for clients that will issue many queries—such as a light client synchronizing multiple wallet addresses.

The client maintains a limited *query budget* of approximately $M = N/T$ queries per hint set, where N is the database size and $T = \lfloor \sqrt{2N} \rfloor$. Once the budget is exhausted, the client must re-download hints.

5.2 Offline Phase: Hint Generation

In the offline phase, the hint server computes a compact representation of the database that allows the client to later recover individual entries using only $O(\sqrt{N})$ work per query.

Setup. Let $\text{DB}[0], \dots, \text{DB}[N-1]$ denote the database entries, each of w bytes. Let $P : [N'] \rightarrow [N']$ be a pseudorandom permutation over a padded domain of size $N' \geq N$ (chosen so that $2N' \bmod T = 0$). Define the parameters:

$$\begin{aligned} T &= \lfloor \sqrt{2N} \rfloor, \\ M &= N'/T \quad (\text{number of segments}). \end{aligned}$$

Hint computation. The server computes M hint values, each of w bytes, via a scatter-XOR operation:

$$H[s] = \bigoplus_{\substack{k \in [N] \\ \lfloor P(k)/T \rfloor = s}} \text{DB}[k], \quad s \in [M].$$

That is, each database row k is assigned to segment $s = \lfloor P(k)/T \rfloor$ by the PRP, and the hint for segment s is the XOR of all rows assigned to it. Rows in the range $[N, N')$ are virtual zero-rows and do not affect the XOR. The total hint size is $M \times w$ bytes per group.

Per-group hints. The system maintains 155 independent HarmonyPIR instances—one per PBC group (75 index-level + 80 chunk-level). Each instance uses a distinct PRP key derived from a shared master key by XORing the group identifier into bytes 12–15 of the key. The hint server computes and streams hints for all 155 groups, using batch parallelism for efficiency.

5.3 Online Phase: Querying

To query for a database entry at index q , the client proceeds as follows:

1. **Locate via PRP:** Compute the cell $c = P(q)$. Determine the segment $s = \lfloor c/T \rfloor$ and position $r = c \bmod T$.
2. **Build request:** Construct the set of indices to request from the query server—namely, all database rows that the PRP maps into segment s , *except* for the target row q itself. Formally, the request set is:

$$R = \{P^{-1}(s \cdot T + j) : j \in [T], j \neq r\} \cap [N].$$

The client sends R (as sorted u32 indices) to the query server.

3. **Server response:** The query server returns the XOR of the requested entries: $\bigoplus_{i \in R} \text{DB}[i]$.
4. **Recover answer:** The client XORs the server's response with the stored hint $H[s]$:

$$\text{DB}[q] = H[s] \oplus \bigoplus_{i \in R} \text{DB}[i].$$

This works because $H[s]$ is the XOR of *all* rows in segment s (including $\text{DB}[q]$), and the server returned the XOR of all rows *except* $\text{DB}[q]$.

5. **State update (relocation):** After recovering $\text{DB}[q]$, the client updates its local data structure so that future queries remain correct. This involves “relocating” row q within the hint structure so that the same segment is not queried twice for the same position.

Each online query involves sending $O(T) = O(\sqrt{N})$ indices and receiving w bytes back. The query server's work is also $O(T \cdot w)$ —a linear scan over roughly $\sqrt{N}$ entries—which is sublinear in the database size N .

5.4 Dummy Queries

To prevent the server from distinguishing real queries from padding, the client generates *synthetic dummy* queries for PBC groups that do not hold a real query in a given round.

A synthetic dummy sends exactly $T - 1$ sorted distinct u32 indices drawn uniformly at random from $[0, \text{real}_n)$, where real_n is the group's database size. The resulting request has the same format and length as a real query but does not modify the client's hint state; the corresponding response entries are discarded via XOR-cancellation during response processing. Sending a fixed count of indices—identical to the count in every real query—closes the per-group request-count side channel that would otherwise allow the server to distinguish the real-query slot from padding dummies by tracking which group's request size drifts as hints are consumed.

5.5 Integration with Batch PIR

HarmonyPIR plugs into the same PBC framework described in Section 3. The client maintains 155 independent HarmonyPIR group instances:

- **Index level:** $K=75$ groups, each with entry size $w = 52$ bytes (one cuckoo bin = 4 slots $\times$ 13 bytes per index slot).
- **Chunk level:** $K_{\text{chunk}}=80$ groups, each with entry size $w = 132$ bytes (one cuckoo bin = 3 slots $\times$ 44 bytes per chunk slot).

The query protocol proceeds in rounds, analogous to DPF-PIR:

1. The client places m ScriptPubKey queries into K PBC groups via cuckoo hashing (same as Section 3).
2. For each of the 2 cuckoo hash function rounds at the index level:
 - Groups with a real query generate a HarmonyPIR request targeting the assigned cuckoo bin.
 - Groups without a real query generate a synthetic dummy.
3. All K requests are sent to the query server in a single batch. The server responds with K XOR results.
4. The client processes responses: real queries recover index slots via the hint XOR; dummy responses are discarded.
5. The process repeats for the chunk level with K_{chunk} groups.

5.6 PRP Backend Selection

The PRP P must be a permutation over a domain of size N' (typically $\sim 10^6$), which is far smaller than the 2^{128} domain of standard block ciphers like AES. Several small-domain PRP constructions are available, each with different performance characteristics:

- **HMR12 (swap-or-not) [HMR12]:** A Feistel-like construction based on card shuffles. Uses AES round functions with 4-way pipelining. This is the default backend and serves as the baseline.
- **FastPRP:** A radix-sort-based PRP with $O(N \log N)$ batch evaluation. Precomputes the entire permutation table, trading memory for speed. Achieves 2–4 $\times$ speedup over HMR12.
- **ALF [AOP25]:** An AES-NI-based format-preserving encryption scheme optimized for SIMD execution. Achieves the best performance: 4–8 $\times$ faster than HMR12. Requires hardware AES-NI support (available on modern x86 and ARM processors, and via WASM SIMD128 in browsers).

The PRP backend is selected at session initialization and must be consistent between the hint server and client. Server-side benchmarks on a 24-thread machine show ALF computing all 75 index hints in ~ 711 ms, compared to ~ 1.05 s for FastPRP and ~ 7.24 s for HMR12.

5.7 Hint Caching and Query Budget

Each HarmonyPIR group instance supports approximately $M \approx N/T \approx \sqrt{N/2}$ queries before its hints are exhausted, where N here refers to the number of cuckoo bins in the group’s table. For the current main database this gives roughly 530 queries per group at the INDEX level (where $N \approx 565,000$ bins, DPF domain 2^{20}) and roughly 730 queries per group at the CHUNK level (where $N \approx 1,064,000$ bins, DPF domain 2^{21}). Both levels comfortably exceed typical wallet query volumes.

To avoid re-downloading hints unnecessarily, the client caches hint state:

- **State serialization:** Each group’s full state (hints, PRP key, relocation structure, query counter) can be serialized and stored in the browser’s `localStorage` or on disk.
- **Exhaustion detection:** Before each batch query, the client checks the minimum remaining query budget across all active groups. If any group is exhausted, the client triggers a hint refresh from the hint server.
- **Resilient reconnection:** If the WebSocket connection drops mid-session, the client reconnects with exponential backoff and resumes using cached state, avoiding a full hint re-download.

6 1-Server PIR via OnionPIR

This section describes the single-server PIR instantiation based on OnionPIR [MCR21] and its successor OnionPIRv2 [CR25], a fully homomorphic encryption (FHE) based protocol. Unlike DPF-PIR and HarmonyPIR, OnionPIR requires only *one* server and provides computational privacy under lattice hardness assumptions, eliminating the need for a non-colluding server pair. The trade-off is higher communication cost and server computation per query.

6.1 Overview

OnionPIR uses the composition of two somewhat homomorphic encryption (SHE) schemes—BFV-style encryption and GSW-style encryption—to achieve compact responses. The client encrypts its query index under an FHE scheme, the server homomorphically evaluates a selection function over the encrypted query and the database, and the client decrypts the result to obtain the desired entry.

The protocol is *stateless*: the client generates fresh encryption keys per session and requires no hints or prior database knowledge. The FHE parameters used in our instantiation are:

- Polynomial degree: $d = 2048$
- Plaintext modulus: $p = 32,771$ (a 15-bit prime)
- Entry size: $d \times 15/8 = 3840$ bytes (fixed by the FHE parameters)

The fixed 3840-byte entry size is a fundamental constraint of OnionPIR’s encoding: each database entry occupies exactly one RLWE plaintext polynomial, with 15 usable bits per coefficient across 2048 coefficients.

6.2 Database Layout

Because OnionPIR’s entry size (3840 bytes) is much larger than the DPF/HarmonyPIR entry sizes (39 or 132 bytes), we use a different database layout that packs multiple logical records into each 3840-byte OnionPIR entry.

6.2.1 Index Level (75 Groups)

Each PBC group’s index database is organized as a cuckoo hash table where each bin is packed with up to 256 index slots, fitting exactly into one 3840-byte entry:

- **Slot format:** 15 bytes per slot—8-byte fingerprint tag, 4-byte entry identifier (pointing into the main database), 2-byte byte offset within the entry, and 1-byte entry count.
- **Bin packing:** $256 \times 15 = 3840$ bytes per bin, matching one OnionPIR entry.
- **Cuckoo design:** 2 hash functions over $\sim 8,840$ bins per group.

The client computes both candidate bin indices, issues one OnionPIR query per candidate, and scans the returned 256 slots for a matching fingerprint tag.

6.2.2 Main Database (80 Groups)

UTXO data is greedily packed into 3840-byte entries:

- Multiple addresses share a single entry via greedy bin-packing. If an address’s serialized UTXO data does not fit in the remaining space of the current entry, padding is inserted and a new entry begins.
- From approximately 53.6 million addresses, the packing produces $\sim 815,000$ unique entries with $\sim 95.9\%$ packing efficiency.
- Each entry is assigned to 3 of the 80 PBC groups (triplication for batch PIR).

- Within each group, entries are placed into a cuckoo hash table with 6 hash functions, 1 slot per bin, and $\sim 32,000$ bins per group.

The client computes the 6 candidate bin indices in each assigned group and determines which bin holds its target entry by performing client-side cuckoo table reconstruction.

6.3 NTT Preprocessing and Storage

The server preprocesses each database into Number Theoretic Transform (NTT) form for efficient FHE evaluation. This is a one-time offline step.

Expansion factor. Each 3840-byte logical entry expands to 16,384 bytes in NTT form, a $4.27\times$ expansion.

Shared NTT store. Since the same packed entry appears in multiple PBC groups (triplication), we avoid redundant storage by maintaining a single shared NTT store indexed by entry identifier, plus per-group indirection tables that map cuckoo bins to entries in the shared store. This reduces total storage:

- **Without deduplication:** ~ 51 GB (all groups store independent NTT copies).
- **With shared store:** ~ 24 GB—comprising ~ 13 GB for the shared main-database NTT store, ~ 10.7 GB for per-group index NTT databases (no deduplication possible since each group’s bins are unique), and ~ 13 MB for indirection tables.

Level-major layout. The shared NTT store uses a level-major memory layout: all entries’ coefficients for NTT level ℓ are stored contiguously, so that a single query reads within a ~ 6.5 MB contiguous slab. This provides good cache and TLB behavior compared to the entry-major alternative, which would cause 16,384-byte strides and severe cache thrashing.

All NTT data is memory-mapped via `mmap2` for fast server startup without loading the entire dataset into RAM.

6.4 Query Protocol

A complete OnionPIR session proceeds as follows:

Key exchange. The client generates FHE keys and registers them with the server:

- Galois keys: ~ 274 KB (for automorphism operations).
- GSW keys: ~ 324 KB (for homomorphic multiplication).
- Total key material: ~ 598 KB, registered once per session.

The server deserializes keys into a shared key store, enabling reuse across all 155 OnionPIR instances (75 index + 80 chunk) without redundant deserialization.

Index queries. For each of the 75 PBC groups, the client generates up to 2 FHE-encrypted queries (one per cuckoo hash function), targeting specific bins in the group’s index table. Each query ciphertext is ~ 16 KB (seed-compressed). The server homomorphically evaluates the selection function and returns a ~ 14 KB encrypted response containing one 3840-byte entry. The client decrypts and scans the 256 index slots for a matching fingerprint.

Chunk queries. Using the entry identifiers from the index phase, the client issues one FHE query per PBC group over the main database. The server evaluates using the shared NTT store with indirection and returns encrypted 3840-byte entries. The client decrypts and extracts UTXO data at the known byte offset.

Communication summary:

Phase	Upload	Download	Queries
Key exchange	598 KB	—	1
Index	$75 \times 2 \times 16 \text{ KB} \approx 2.4 \text{ MB}$	$75 \times 2 \times 14 \text{ KB} \approx 2.1 \text{ MB}$	≤ 150
Chunks	$80 \times 16 \text{ KB} \approx 1.3 \text{ MB}$	$80 \times 14 \text{ KB} \approx 1.1 \text{ MB}$	80
Total	$\sim 4.3 \text{ MB}$	$\sim 3.2 \text{ MB}$	≤ 231

6.5 Database Generation Pipeline

The OnionPIR database generation is a three-stage pipeline, separate from the DPF/HarmonyPIR pipeline due to the different entry sizes:

1. **UTXO packing** (`gen_1_onion`): Groups UTXOs by ScriptPubKey, serializes each group with VarInt encoding, and greedily packs them into 3840-byte entries. Produces the packed entry file and a 27-byte-per-address index file (20-byte script hash, 4-byte entry ID, 2-byte byte offset, 1-byte entry count).
2. **Chunk database construction** (`gen_2_onion`): For each of the 80 PBC groups, builds a 6-hash cuckoo table mapping entry IDs to bins, populates an OnionPIR server instance, and preprocesses into NTT form. The shared NTT store is written in level-major layout. This is the most expensive step (~ 81 seconds on a modern machine).
3. **Index database construction** (`gen_3_onion`): For each of the 75 PBC groups, builds a 2-hash cuckoo table where each bin packs 256 index slots into one 3840-byte entry, preprocesses into NTT form, and saves to disk (~ 14 seconds total).

6.6 Performance

Server computation. On a multi-core machine, each OnionPIR query evaluation takes ~ 12 ms for index-level queries (8,256 padded entries per group) and ~ 20 ms for chunk-level queries (43,264 padded entries per group). A full batch requires evaluating 150 index queries and 80 chunk queries sequentially, for a total server compute time of ~ 2.5 seconds per batch.

Noise budget. After decryption, the remaining noise budget is 4 bits for index-level queries and only 2 bits for chunk-level queries. The chunk-level budget is tight and warrants monitoring as the UTXO set grows, as larger databases may require adjusting the FHE parameters.

End-to-end latency. At 50 Mbps broadband, a complete UTXO lookup (key exchange + index + chunk phases) takes approximately 3.5 seconds, dominated by server computation rather than network transfer.

7 Integrity via Per-Group Merkle Trees

All three PIR backends described so far provide *privacy*: neither a single server (OnionPIR) nor a pair of non-colluding servers (DPF-PIR, HarmonyPIR) learns which address a client queried. Privacy alone, however, does not guarantee that the returned UTXO data is correct. A malicious server could silently substitute entries and the client would never notice. Worse, a carefully chosen substitution pattern could implement a *selective failure attack* (Section 1.2) that leaks information about the query through the client’s later behavior.

This section describes the integrity layer that we build on top of the PIR protocols. The construction is a family of per-group SHA-256 Merkle trees over the cuckoo bins of the live database; verification is itself driven through the same PIR primitives used for data retrieval, so integrity inherits the privacy of the underlying protocol.

7.1 Design Goals

We deliberately avoid schemes that depend on a trusted setup or on asymmetric cryptography (e.g., KZG [KZG10]), for three reasons. First, post-quantum security: the underlying PIR protocols are already post-quantum (DPF, HarmonyPIR) or based on lattice hardness (OnionPIR), and we would like integrity to match. Second, parameter agility: a new checkpoint is published roughly every few hours, and recomputing a KZG commitment over the entire database at each update would be expensive and painful to re-distribute. Third, reusability: we prefer a construction that needs nothing from the client beyond the primitives it already has—a hash function and the ability to issue PIR queries. SHA-256 Merkle trees satisfy all three requirements.

We also need integrity that is compatible with *batch PIR*. The natural place to verify an entry is at the cuckoo-bin level, because a single PIR query retrieves exactly one bin per PBC group. We therefore build a Merkle tree whose leaves are cuckoo bins, not individual UTXO records. This keeps the verification proof at the same granularity as the PIR response, avoiding any correlation between what was verified and what was actually accessed.

7.2 Construction

Recall from Section 2 that our data layout is organized into two PIR levels (INDEX and CHUNK), each sharded across a fixed number of PBC groups ($K=75$ for INDEX, $K_{\text{chunk}}=80$ for CHUNK). Each group is a separate cuckoo hash table whose rows (“bins”) are the atomic unit retrieved by one PIR query.

We build an independent Merkle tree for each group, giving $K + K_{\text{chunk}} = 155$ trees in total. Every tree has arity $A = 8$ and covers all B_g bins of its group.

Leaf hashing. For bin index i in group g , the leaf hash is

$$L_{g,i} = \text{SHA256}(\text{u32le}(i) \parallel \text{bin}_{g,i}),$$

where $\text{bin}_{g,i}$ is the raw byte contents of the bin ($\text{slots_per_bin} \cdot \text{slot_size}$ bytes) and $\text{u32le}(i)$ is i encoded as a little-endian 32-bit integer. Prepending the index prevents a server from swapping bins between positions.

Internal nodes. Internal nodes combine $A = 8$ children: if $c_0, \dots, c_7$ are the 32-byte child hashes (padded with 0³² if fewer), the parent is $\text{SHA256}(c_0 \parallel c_1 \parallel \dots \parallel c_7)$. A higher arity means fewer levels: a group with $\sim 10^6$ bins has only $\lceil \log_8 10^6 \rceil = 7$ levels, so a single Merkle proof costs roughly 7 sibling fetches.

Per-group root. The root of group g ’s tree is R_g , a 32-byte hash.

Super-root. Concatenating all 155 per-group roots in a canonical order (75 index roots followed by 80 chunk roots) and hashing with SHA-256 yields a single 32-byte *super-root* that commits the entire database:

$$R^* = \text{SHA256}(R_1 \parallel R_2 \parallel \dots \parallel R_{155}).$$

The super-root, together with the database version, is what clients compare across servers or against an out-of-band trust anchor. One 32-byte value covers every bin of every group at both levels.

7.3 Storage Layout

Merkle sibling hashes are stored on the server in files that are themselves structured as PBC-queryable tables, so that the client can fetch proofs through ordinary PIR queries.

Flat sibling tables. For each table type (INDEX or CHUNK) and each tree level ℓ , we maintain a single file whose row (g, j) stores all $A = 8$ children of the j -th internal node in group g 's level- ℓ nodes. Each row is exactly $A \cdot 32 = 256$ bytes. This is the “flat” layout: sibling data is indexed directly by group id and node position, with no cuckoo hashing or tag matching—the client already knows the position and group of every sibling it needs.

Tree-top cache. The top of every group's tree (levels close to the root, where there are only a handful of nodes) is cached server-side in a single blob, `merkle_bucket_tree_tops.bin`, along with all 155 per-group roots and the super-root. For the current database this blob is about 9.1 MB.

Total footprint. The dominant cost is the sibling tables: about 4.6 GB for the full database. (An earlier variant of this scheme used global Merkle trees and required roughly 14.8 GB; partitioning per group and switching to a flat layout cut sibling storage by more than $3\times$.) All sibling data is memory-mapped and never loaded in full.

7.4 Verification Flow

A client that has just retrieved a bin via DPF-PIR or HarmonyPIR verifies it as follows.

1. **Anchor the tree-top cache.** On the first verification of a session, the client fetches the tree-top cache and computes its SHA-256. The expected hash is included in the top-level server metadata response (Section 11.4); if it matches, the client has a locally trusted copy of all per-group roots and of the super-root.
2. **Compute the leaf.** For the retrieved bin at index i in group g , the client computes $L_{g,i}$ as above. This is the Merkle leaf it needs to connect to the per-group root R_g .
3. **Walk the tree, one level at a time.** For each sibling level $\ell = 0, 1, 2, \dots$, the client needs the $A - 1 = 7$ sibling hashes of its current node at that level. Because siblings are stored in the flat level- ℓ table indexed by (group, node position), the client already knows exactly which row of which file it needs to read—but reading that row in cleartext would reveal the query. Instead, the client treats each sibling table as just another PIR database and issues a batch DPF-PIR query against it, piggybacking on the regular 2-server DPF primitive.
4. **Combine and climb.** Using the retrieved row of 8 child hashes, the client replaces the slot at its own position with its current hash, hashes the concatenation with SHA-256, and advances to the next level with $\lfloor i/A \rfloor$ as the new node index. After the final level it reaches a hash that should be present in the cached tree-top for group g ; if so, the leaf is successfully connected to R_g .
5. **Check against the super-root.** Because R_g was part of the tree-top blob whose SHA-256 the client verified in step 1, reaching R_g closes the proof. Equivalently, the client has shown that the retrieved bin is included in the committed database under the known super-root.

Because every sibling fetch is itself a DPF-PIR query, the server learns nothing about which bin was being verified beyond what it already learned from the data query—and because DPF-PIR is information-theoretic, that “learned nothing” is exactly zero. Verification therefore does not weaken the privacy guarantees of the data-retrieval phase.

Batching. In practice the client verifies *all* addresses in a batch at once. For each sibling level, the client places one DPF-PIR query per address into a K -group PBC structure (with synthetic dummies for unused groups, exactly as in the main protocol), so that a single round of the Merkle walk verifies every address in parallel. The communication cost of Merkle verification is one DPF-PIR batch per sibling level (~ 7 levels for INDEX and CHUNK) plus a single tree-top fetch that is amortized across the whole session.

7.5 Integration with the Protocols

DPF-PIR. Both data queries and Merkle sibling queries use the same DPF-PIR primitive and are issued against the same pair of servers. A full *verified* DPF-PIR batch thus consists of: (i) the INDEX-level batch, (ii) the CHUNK-level

batch, (iii) one tree-top fetch, and (iv) one DPF-PIR sibling batch per Merkle level. In our measurements the Merkle verification adds a small constant number of extra rounds and does not dominate total latency.

HarmonyPIR. Per-bucket Merkle integrates cleanly because the HarmonyPIR data response already retrieves a full cuckoo bin, which is exactly the Merkle leaf unit. The sibling tables themselves are verified via DPF-PIR regardless of which backend was used for the primary data query: we use DPF as the uniform integrity primitive because sibling tables are small compared to the main UTXO tables, and layering a second stateful protocol on top would add engineering surface area with no real benefit.

OnionPIR. OnionPIR uses a separate per-bin Merkle design with two flat trees (one for the INDEX level, one for CHUNK). Leaf hashes are computed over decrypted bin contents, and sibling queries are issued through the same FHE primitive used for data retrieval. The underlying two-tree structure mirrors the two-level PIR architecture, and the privacy invariants (batch padding, per-query leaf-count symmetry) are preserved through the same mechanisms described above.

Delta databases. The sibling tables, tree-top cache, super-root, and catalog flag are per-database, so the scheme extends to delta databases (Section 8) without any protocol changes—a client issues its sibling requests with the same `db_id` byte that routed its data query. Building Merkle data for small delta databases is optional and deferred by default, controlled by the `has_bucket_merkle` field in the catalog. (The flag’s name predates the terminology unification: in implementation files, all per-group Merkle artifacts are named `merkle_bucket_*`.)

7.6 Build Pipeline

The sibling tables, tree-tops, per-group roots, and super-root are produced by a dedicated build step that runs after the cuckoo tables are assembled. For each group, the step walks the cuckoo bins, hashes leaves, and emits one flat sibling file per level. The step is embarrassingly parallel across the 155 groups and across levels, and takes a few minutes on a modern multi-core server for the full UTXO database. Because Merkle data is an additive layer on top of the cuckoo tables, generating or skipping it is a runtime choice—a freshly built database is immediately usable for PIR queries and acquires verifiability as soon as the Merkle step completes.

8 Incremental Sync via Delta Databases

A light client that has already synchronized up to some height h_A and then comes back later wants to learn which of its UTXOs have changed since h_A , not to re-download the entire snapshot. We support this with *delta databases*: a new kind of PIR-queryable database that encodes only the differences between two consecutive checkpoints.

8.1 What a Delta Contains

Given two checkpoint heights $h_A < h_B$, the delta $\Delta_{A \rightarrow B}$ records, for each script hash s whose set of spendable outputs changed in $(h_A, h_B]$:

- The set of *spent* UTXOs belonging to s : references (TXID, vout) to outputs that existed at h_A and no longer exist at h_B . Amounts are not included because the client already learned them when it synced to h_A .
- The set of *new* UTXOs belonging to s : full records (TXID, vout, amount) for outputs that did not exist at h_A and exist at h_B .

A script hash that saw no change simply does not appear in the delta; a query for such an address returns an empty result. A client that retrieves $\Delta_{A \rightarrow B}$ for all of its addresses can mechanically update its local UTXO set from state h_A to state h_B : remove the spent entries, add the new ones.

8.2 Wire Format

The per-entry payload returned by a delta PIR query has the following layout, where varint denotes an LEB128 unsigned varint:

$$\begin{aligned} & [\text{varint } n_{\text{spent}}] \\ & \quad [32\text{B txid, varint vout}] \times n_{\text{spent}} \\ & [\text{varint } n_{\text{new}}] \\ & \quad [32\text{B txid, varint vout, varint amount}] \times n_{\text{new}}. \end{aligned}$$

This is a straightforward extension of the full-database payload format from Section 2: a full-database entry is a list of (TXID, vout, amount) triples preceded by a count, and a delta entry is a pair of such lists, one for spent and one for new, where the spent list omits the amount field. Using LEB128 varints keeps the payload small for the common case where n_{spent} and n_{new} are both single digits.

8.3 Storage and PIR Layout

A delta database is, structurally, just another instance of the same two-level cuckoo layout used by the main UTXO database (Section 2): an INDEX table that maps script hashes to chunk locations and a CHUNK table that stores the actual delta payloads. The cuckoo parameters (number of PBC groups K , K_{chunk} ; slots per bin; slot size) are identical to the main database, which means that every piece of build machinery, server code, and client code that already works on the main database works on a delta database with no protocol changes.

Delta databases are typically much smaller than full snapshots: for a delta covering a few thousand blocks, the number of affected script hashes is on the order of a few million at most, versus tens of millions for the full set. Smaller tables mean smaller DPF domains (lower `dpf_n`), which in turn means cheaper per-query server work and smaller Merkle sibling tables when Merkle data is built.

8.4 Build Pipeline

Delta construction is a diff of two UTXO snapshots:

1. **Compute the raw diff.** Walk the two snapshots and emit, per script hash, the sets of UTXOs present in exactly one of them. A UTXO present only at h_A is spent; a UTXO present only at h_B is new. The same dust filter used for full snapshots is applied to both sides (Section 2.4).
2. **Pack delta records into chunks.** For each script hash, serialize its spent and new lists with the varint format above, and greedily pack the resulting records into fixed-size chunks using the same chunking code as the main database.
3. **Cuckoo-hash the INDEX and CHUNK tables.** Use the same `build_cuckoo_generic` pipeline as the main database.
4. **(Optional) Build per-group Merkle data.** Run `gen_4_build_merkle_bucket` on the delta tables to produce sibling files, tree tops, per-group roots, and a super-root, exactly as for the main database (Section 7).

The result is a directory of files that the server can memory-map and register as a new database at any `db_id` via its `databases.toml` configuration.

8.5 Serving Deltas

Because a delta is structurally the same shape as the main database, the server’s DPF, HarmonyPIR, OnionPIR, and Merkle handlers all work on it unchanged. The only thing that varies is *which* database the server should read from to answer a given request. This is controlled by the `db_id` byte described in Section 11.4: a DPF batch query tagged with `db_id=7`, for example, is routed to the seventh loaded database, which might be a specific delta. Merkle sibling and tree-top requests carry the same byte, so a verified delta query verifies against that delta’s own per-bucket Merkle tree.

8.6 Incremental Sync Workflow

A complete incremental sync from local height h_{last} to the server’s tip h_{tip} proceeds as follows:

1. The client fetches the database catalog (Section 11.4) and computes a sync plan.
2. For each step of the plan (either a full checkpoint or a delta), the client runs its standard batch PIR pipeline against the corresponding `db_id`, retrieving results for all of its addresses.
3. For a delta step, the client applies the decoded spent/new sets to its local UTXO state; for a full step, it replaces its local state with the retrieved snapshot.
4. If per-group Merkle is available for that database, the client verifies the retrieved bins against the delta’s own super-root (which the client in turn compares against whatever out-of-band anchor it trusts) before applying the updates.

The end result is that a returning client does not need to re-download its entire UTXO view. In the common case where only a handful of new blocks have been added since the last sync, only a small delta’s worth of PIR work is required.

9 Anonymous Rate Limiting

A PIR server that answers queries without restriction is vulnerable to *query exhaustion*: a malicious client can issue unlimited queries and gradually reconstruct large portions of the database, undermining the very purpose of PIR. Traditional rate limiting—by IP address, API key, or account—forces the client to reveal its identity, destroying the privacy that PIR is designed to provide. This tension between abuse prevention and user anonymity is inherent to any privacy-preserving service.

This section describes two cryptographic mechanisms that resolve the tension: *Anonymous Rate-limited Credentials* (ARC) [IET25] and Cashu’s Blind Authentication (NUT-22) [Cas22]. Both allow a server to enforce a per-client query budget without learning which client is making which queries.

9.1 The Problem: Anonymous Rate Limiting

We want a server to enforce a limit of L queries per authorized client, while satisfying three properties:

1. **Privacy:** The server cannot link a query to the client’s identity, nor can it link two queries from the same client.
2. **Rate enforcement:** A client who has exhausted its budget cannot produce a valid query that the server will accept.
3. **Unforgeability:** A client without a valid authorization cannot produce a query that the server will accept.

The solution is to separate the protocol into two phases: an *authorization phase*, where the client proves its right to access the service (possibly revealing its identity once), and a *query phase*, where the client presents anonymous credentials derived from that authorization with each batch of PIR queries. The server verifies the credential, deducts from the budget, and processes the PIR query—all without learning which authorized client performed the query.

9.2 Anonymous Rate-Limited Credentials (ARC)

ARC [IET25] is an IETF Privacy Pass protocol (ciphersuite ARCV1 – P256) that issues a single multi-show credential to a client, which the client can then present up to a predetermined limit L times without the presentations being linkable to each other or to the issuance.

9.2.1 Cryptographic Building Blocks

ARC operates over the P-256 elliptic curve and combines three primitives:

- **Algebraic MAC (MAC_GGM):** The server’s signature on the blinded credential is an algebraic message authentication code rather than a simple Diffie–Hellman exponentiation. This enables the server to embed structured data (the rate limit) into the credential.
- **Schnorr proofs:** Both the client (during issuance) and the server (during signing) attach zero-knowledge Schnorr proofs of knowledge of their respective secrets, ensuring that neither party can cheat in the blind exchange.
- **Per-bit Pedersen range proofs:** During presentation, the client proves that a hidden nonce $\nu \in [0, L)$ without revealing ν . The proof decomposes the nonce bit-by-bit and commits to each bit with a Pedersen commitment, then proves each commitment encodes a bit and that the bits reconstruct a value in range.

9.2.2 Issuance Protocol

The issuance is a three-message blind exchange between the client and an *issuer* (which may be the PIR server itself or a separate authorization service):

1. **Client request.** The client generates a random secret, blinds it together with a `request_context` (e.g., a session identifier or the hash of an out-of-band authorization token), and produces a Schnorr proof of knowledge

of the blinded values. The server receives only blinded data.

2. **Server response.** The server verifies the client's Schnorr proof, performs an out-of-band authorization check (e.g., the client has paid for a subscription or passed a CAPTCHA), and, if authorized, signs the blinded credential with its MAC key. It attaches its own Schnorr proof of correct signing.
3. **Client finalization.** The client verifies the server's proof, unblinds the response, and obtains a plaintext credential. The server never sees the unblinded credential and cannot link it to the issuance session.

The credential embeds a `presentation_limit` L chosen by the server during issuance.

9.2.3 Presentation Protocol (Multi-Show)

To make a query, the client creates a presentation from the credential, a `presentation_context` (binding the presentation to the specific query or epoch), and a nonce ν that increments from 0 to $L - 1$:

1. The client generates a per-bit Pedersen range proof showing $\nu \in [0, L)$ without revealing ν .
2. The client produces a Schnorr proof of knowledge of the credential.
3. The server verifies both proofs and computes a deterministic *tag* from $(\text{credential}, \nu, \text{presentation_context})$.
4. The server checks whether this tag has been seen before in the current presentation context; if so, it rejects the request (double-spend detection).
5. If the tag is fresh, the server stores it and processes the query.

After L valid presentations, the client's internal nonce counter reaches L and the credential is exhausted. Any attempt to present with $\nu \geq L$ fails the range proof. Any attempt to reuse a nonce fails the tag uniqueness check.

9.2.4 Integration with PIR

For each batch of PIR queries, the client attaches a fresh ARC presentation. The server verifies the presentation before evaluating any DPF keys or HarmonyPIR requests. If verification fails (exhausted credential, invalid proof, duplicate tag), the server drops the connection without processing the PIR payload.

Because the presentation is zero-knowledge and the tag is a deterministic function of hidden values, the server learns only that the request carries a valid, unexhausted credential—not which credential, which client, or which previous queries belong to the same credential. The rate limit is enforced cryptographically without deanonymization.

9.3 Cashu Blind Authentication (NUT-22)

Cashu [Cas22] is a Chaumian ecash system for Bitcoin built on blind signatures over secp256k1. Its Blind Authentication extension (NUT-22) adapts the same ecash primitives to provide anonymous authentication tokens.

9.3.1 Cryptographic Building Blocks: BDHKE

Cashu's core blind signature scheme is the Blind Diffie–Hellman Key Exchange (BDHKE) over secp256k1. The mint holds a private key k and publishes public key $K = kG$. To obtain a blind signature on a secret message x :

1. The client maps x to a curve point $Y = \text{hash_to_curve}(x)$ using a domain-separated hash-to-curve.
2. The client picks a random blinding factor r and computes the blinded message $B' = Y + rG$, sending B' to the mint.
3. The mint signs the blinded point: $C' = kB'$.
4. The client unblinds: $C = C' - rK = kY + krG - krG = kY$.

The pair (x, C) forms an ecash token. The mint can verify $C \stackrel{?}{=} k \cdot \text{hash_to_curve}(x)$ at redemption time but cannot link the unblinded token back to the blinded signing request.

9.3.2 Two-Layer Authentication Flow

Cashu Blind Auth uses a two-layer design:

1. **Clear authentication (NUT-21).** The client authenticates once to an external identity provider (e.g., via OpenID Connect) and obtains a Clear Authentication Token (CAT). The CAT is a bearer token that proves the client passed an authorization check but is itself linkable—every use of the same CAT reveals the same identity.
2. **Blind auth token minting (NUT-22).** The client uses the CAT to mint multiple Blind Authentication Tokens (BATs) from the mint, via the standard BDHKE blind signature protocol. The BATs use a special keyset where the unit is `auth` and only a single amount value is supported. The mint signs blinded BATs, the client unblinds them, and the resulting tokens are indistinguishable from each other to the mint.

Each BAT is a single-show token: presenting it consumes it. The mint adds the token’s secret to its spent list and rejects any subsequent presentation.

9.3.3 Integration with PIR

A client mints a batch of BATs upfront (using a CAT obtained through whatever authorization policy the server requires). Before each PIR query batch, the client attaches a fresh BAT. The server verifies the BAT against its auth keyset and checks that it has not been spent, then processes the PIR request. The BAT is consumed irrespective of whether the PIR query succeeds.

The mint publishes which API endpoints require blind auth and caps the number of BATs that can be minted per CAT request (`bat_max_mint`), providing a server-configurable query budget per authorization event.

9.4 Comparison

Property	ARC	Cashu Blind Auth
Curve	P-256	secp256k1
Core primitive	Algebraic MAC + range proofs	BDHKE blind signatures
Credential type	Multi-show (up to L)	Single-show (one per BAT)
Rate limit enforcement	Cryptographic (range proof + tag)	Ecological (mint a fixed batch of BATs)
Round trips per L queries	1 issuance + L presentations	1 CAT + 1 BAT minting batch + L presentations
Zero-knowledge presentations	Yes (Schnorr + range proof)	No (secret revealed at spend)
Standardization	IETF Privacy Pass draft	Cashu NUT specification

Table 2: Comparison of ARC and Cashu Blind Auth for anonymous rate limiting. ARC uses range proofs to encode the rate limit directly into a single multi-show credential; Cashu issues a batch of single-show tokens.

The two approaches represent different points on the design spectrum. ARC embeds the rate limit *inside* the credential using zero-knowledge range proofs, giving the server a single tag to track per presentation context and requiring no re-issuance until the limit is exhausted. Cashu separates the rate limit from the credential: each BAT authorizes exactly one query, and the rate limit is enforced by the number of BATs the server is willing to issue in response to a single CAT. This makes Cashu’s per-query verification simpler (a single BDHKE signature check) but requires the client to manage a pool of single-use tokens.

Both mechanisms integrate with all three PIR backends (DPF, HarmonyPIR, OnionPIR) without protocol changes: the anonymous credential is verified at the transport or application layer before the PIR handler is invoked, so the PIR server itself need not be aware of the rate-limiting scheme.

10 Protocol Comparison

We compare the three PIR instantiations across five dimensions: trust model, communication cost, server computation, client requirements, and suitability for different usage patterns.

10.1 Trust Models

- **DPF-PIR** requires two servers that do not collude. Privacy is *information-theoretic*: if the servers do not communicate, neither learns anything about the query, regardless of computational power. However, if the two servers collude, the client’s query is immediately revealed. Running one of the two servers inside a TEE (Section 4.3) refines this requirement: the TEE-hosted server’s operator is removed from the trust base because TEE memory encryption and remote attestation prevent the operator from reading DPF shares or tampering with the server binary. The effective trust model becomes one trusted operator plus the TEE vendor’s hardware root of trust, rather than two trusted operators.
- **HarmonyPIR** also requires two non-colluding servers (a hint server and a query server), providing the same information-theoretic privacy guarantee. The hint server and query server need not be the same pair of servers—any two servers that hold the database and do not collude can be used. The same TEE approach applies: running either the hint server or query server inside a TEE removes its operator from the trust base.
- **OnionPIR** requires only a single server. Privacy is *computational*: the query is encrypted under a lattice-based FHE scheme, and the server cannot distinguish queries even with unlimited time, assuming the hardness of the Ring Learning with Errors (RLWE) problem. This eliminates the non-colluding assumption entirely. A TEE provides no additional privacy benefit for OnionPIR since there is no second server to collude with; integrity and correct computation are the remaining motivations for TEE use with OnionPIR.

10.2 Communication Costs

Metric	DPF-PIR	HarmonyPIR	OnionPIR
Trust model	2-server (IT)	2-server (IT)	1-server (comp.)
Offline download	0	600 MB–2 GB	598 KB (keys)
Online upload	~75 KB	~10 KB	~4.3 MB
Online download	~25 KB	~10 KB	~3.2 MB

Table 3: Communication costs per batch query for a typical session querying multiple addresses. DPF-PIR and HarmonyPIR costs are per-server (the client communicates with two servers). OnionPIR costs are for the single server. HarmonyPIR offline cost is a one-time download per session, amortized across all subsequent queries.

DPF-PIR has the smallest per-query communication because DPF keys are compact ($O(\lambda \log N)$ bits). HarmonyPIR achieves even lower online communication at the cost of the large offline hint download. OnionPIR has the highest per-query communication due to FHE ciphertext overhead, but requires no offline phase and no second server.

10.3 Server Computation

DPF-PIR’s server work is a simple XOR scan over the database, making it the fastest in wall-clock time per query despite its linear complexity. HarmonyPIR achieves *sublinear* server work per online query ($O(\sqrt{N})$), at the cost of the offline hint computation. OnionPIR’s server work is dominated by FHE polynomial multiplication, making it the slowest per query.

Metric	DPF-PIR	HarmonyPIR	OnionPIR
Per-query complexity	$O(NB)$	$O(\sqrt{N} \cdot w)$	$O(NB)$ (FHE)
Per-query type	XOR scan	Index lookup	FHE evaluation
Batch time (est.)	~ 0.5 s	~ 0.1 s (online)	~ 2.5 s
Offline server cost	0	$\sim 1\text{--}7$ s (hints)	0

Table 4: Server computation comparison. N is the number of database entries, B the entry size, and w the HarmonyPIR entry width. DPF-PIR’s $O(NB)$ consists of lightweight XOR operations; OnionPIR’s $O(NB)$ involves FHE polynomial arithmetic, which is orders of magnitude more expensive per operation.

10.4 Client Requirements

- **DPF-PIR:** Stateless. The client only needs to generate DPF keys (a few hundred bytes each) and XOR the two server responses. Minimal compute, no persistent state, no WASM dependency.
- **HarmonyPIR:** Stateful. The client must store hint data ($M \times w$ bytes per group, totaling hundreds of megabytes across all 155 groups) and maintain the relocation data structure across queries. Requires a PRP implementation, provided via WebAssembly in the browser. Supports three PRP backends with varying performance.
- **OnionPIR:** Stateless per session (keys are generated fresh), but requires an FHE library for encryption and decryption. The WASM module is ~ 516 KB. The client must also reconstruct cuckoo hash tables locally to determine which bins to query.

10.5 When to Use Which Protocol

The choice of protocol depends on the client’s threat model, query volume, and network conditions:

- **One-time lookups** (e.g., importing a seed phrase with a few addresses): **DPF-PIR** is ideal. No offline cost, minimal communication, and the two-server assumption is reasonable when the client can freely choose two Bitcoin full nodes from the P2P network.
- **Repeated queries** (e.g., a wallet that periodically syncs): **HarmonyPIR** excels once the offline hint download is amortized. Each subsequent query costs only ~ 10 KB of communication and $O(\sqrt{N})$ server work. The hint budget of several hundred queries per group is more than sufficient for typical wallet usage.
- **Maximum privacy** (e.g., the client cannot trust any two servers to not collude): **OnionPIR** is the only option. It requires a single server and provides computational privacy under standard lattice assumptions. The higher communication and computation costs are the price of eliminating the non-colluding assumption.

11 Implementation and Deployment

We have implemented a complete, unified system supporting all three PIR protocols. The codebase consists of server-side components in Rust, browser clients in TypeScript with WebAssembly modules, and a Python plugin for the Electrum Bitcoin wallet. All three protocols share a common data representation, batch PIR framework, and wire protocol.

11.1 System Architecture

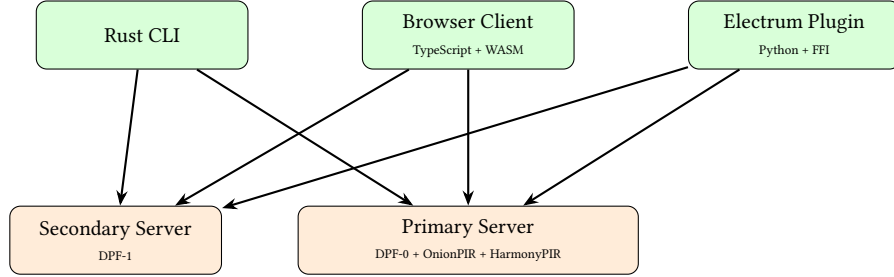


Figure 3: System architecture. A single `unified_server` binary serves all three protocols, deployed as two instances: the primary hosts DPF server 0, OnionPIR, and both HarmonyPIR hint and query roles; the secondary hosts DPF server 1. A client connects to both instances for DPF-PIR, to the primary only for OnionPIR, and to either instance for HarmonyPIR (any two non-colluding nodes work).

Unified wire protocol. All communication uses a compact binary protocol over WebSocket. DPF, HarmonyPIR, and OnionPIR each have their own request variants within the same framing, and each request optionally carries a trailing database identifier byte for multi-database routing (Section 11.4).

Unified server binary. A single Rust binary, `unified_server`, implements all three protocols and selects a role at startup:

- **Primary role:** Serves DPF batch queries (one of the two DPF shares), OnionPIR queries, and both HarmonyPIR hint streaming and HarmonyPIR online queries. Loads every database declared in its configuration file, the shared OnionPIR NTT store, and the per-group Merkle sibling tables. All databases and NTT data are memory-mapped via `mmap2`, so server startup is fast even for multi-gigabyte datasets. OnionPIR evaluation uses the SEAL library [Mic24] via Rust FFI. HarmonyPIR hint generation uses `rayon` for batch parallelism with the selected PRP backend (HMR12, FastPRP, or ALF).
- **Secondary role:** Serves the other DPF share only. A client connects to one primary and one secondary for the 2-server DPF-PIR protocol, ensuring that the two servers do not need to trust each other.

A production deployment typically runs one primary and one secondary on separate hosts, both exposing WebSocket endpoints. Because HarmonyPIR’s hint and query roles can be split across any two non-colluding nodes, a client is free to pick any available primary as its HarmonyPIR query server and another as the hint server. For OnionPIR, the single server can be either instance, since OnionPIR does not rely on non-collusion.

11.2 Browser Deployment via WebAssembly

The browser client supports all three protocols through WebAssembly [W3C19] modules compiled from Rust:

HarmonyPIR WASM. Three independent WASM builds provide the PRP backends:

- `harmonypir/`: HMR12 (swap-or-not) backend—smallest binary, no SIMD requirement.
- `harmonypir-fastprp/`: FastPRP backend—precomputed permutation tables.

- `harmoniypir-alf/`: ALF backend with WASM SIMD128—fastest in browsers with SIMD support (4–8× over HMR12).

Each WASM module exports a `HarmonyGroup` class (whose original name in the codebase, `HarmonyBucket`, predates the bucket-to-group rename) with methods for hint loading, request building, response processing, and state serialization.

OnionPIR WASM. A ~516 KB WASM module provides FHE key generation, query encryption, and response decryption. The client’s secret key can be exported and reused across different OnionPIR instances with different database sizes, avoiding redundant key generation.

Web Workers. For HarmonyPIR, the client distributes the 155 group instances across 4 Web Workers, each running its own WASM instance. This parallelizes the request-building and response-processing phases, providing up to 4× speedup on multi-core devices. Workers communicate via `Transferable ArrayBuffers` to avoid serialization overhead.

DPF-PIR in the browser. DPF key generation and cuckoo hashing are implemented in pure TypeScript without WASM, as the operations (AES-based PRG expansion and XOR accumulation) are lightweight enough for JavaScript.

11.3 Electrum Wallet Integration

We provide a plugin for the Electrum Bitcoin wallet [Voe11] that replaces its native synchronization mechanism with PIR-based UTXO discovery.

Plugin architecture. The plugin hooks into Electrum’s wallet lifecycle:

1. On wallet open, the plugin creates a `PirSynchronizer` that replaces Electrum’s default `Synchronizer` (which sends script hashes to an Electrum server in the clear).
2. The synchronizer collects all wallet addresses, converts them to RIPEMD-160 script hashes, and performs batch PIR queries to discover UTXOs.
3. Discovered UTXOs are written back to the wallet’s internal state, enabling normal spending operations.

Protocol selection. Users choose between the three PIR backends via a Qt settings dialog:

- “DPF 2-Server”: Pure Python DPF implementation with WebSocket transport.
- “HarmonyPIR 2-Server”: Uses PyO3 FFI bindings to call the Rust HarmonyPIR library natively, providing near-Rust performance for PRP evaluation and state management.
- “OnionPIRv2 1-Server”: Uses `ctypes` FFI to a pre-built native library (`libonionpir`) for FHE operations.

Shared utilities. All three backends share Python implementations of cuckoo hashing, PBC group derivation, `VarInt` decoding, and UTXO data parsing—mirroring the Rust and TypeScript implementations to ensure cross-platform consistency.

11.4 Database Catalog and Client Sync

A single unified server can host multiple logically distinct databases at once. The canonical deployment combines one *full* checkpoint (a complete UTXO snapshot at some block height) with a family of *delta* databases that encode the diffs between successive checkpoints (Section 8). The server loads all databases at startup from a `databases.toml` configuration file that declares, for each entry, a name, type (`full` or `delta`), base and tip block heights, on-disk path, and a warmup priority.

Wire-level routing. Each loaded database is assigned a small integer `db_id`, with the main checkpoint at `db_id=0`. To query a specific database, the client appends a one-byte `db_id` to its `INDEX_BATCH/CHUNK_BATCH` request. Requests without the trailing byte are routed to `db_id=0`, preserving backward compatibility with single-database clients. The same routing is used for Merkle sibling and tree-top requests.

Catalog endpoint. A dedicated catalog request returns the full list of databases currently served. Each catalog

entry contains the `db_id`, type, base height, tip height, cuckoo parameters (index and chunk bin counts, tag seed), and a flag indicating whether per-group Merkle verification data is available for that database. The top-level server metadata response additionally embeds per-database Merkle metadata (arity, level sizes, per-group roots, super-root, and tree-top hash), so that a client can verify Merkle proofs against the correct database without a second round trip.

Client sync planning. Given the catalog and the client’s last synced height h_{last} , the client computes a *sync plan*: an ordered sequence of database queries that advances from h_{last} to the latest available tip. The plan is chosen by the following rule:

1. If h_{last} is absent or 0, pick the highest-tip full checkpoint as a single-step fresh sync.
2. If h_{last} already equals the latest tip, the plan is empty.
3. Otherwise, search for a chain of deltas whose base/tip heights connect h_{last} to the tip. The search is breadth-first over the delta graph, capped at a configurable maximum chain length (currently 5).
4. If no suitable chain exists, or it exceeds the cap, fall back to the latest full checkpoint.

The sync plan is a pure function of the catalog and h_{last} : it performs no I/O and can be computed entirely on the client. The client then executes each step by replaying its standard DPF/HarmonyPIR/OnionPIR query pipeline against the appropriate `db_id`.

11.5 Practical Considerations

Server deployment. Production servers are deployed behind reverse proxies (e.g., nginx) with TLS termination via Cloudflare Tunnel, providing WSS (WebSocket Secure) transport without exposing server IP addresses.

Database updates. When the Bitcoin UTXO set changes (approximately every 10 minutes with each new block), the server operator re-runs the database generation pipeline for the relevant checkpoint and, optionally, publishes a new delta keyed to the previous checkpoint. For DPF-PIR and HarmonyPIR, an update only affects the cuckoo hash tables and sibling/tree-top files. For OnionPIR, the NTT preprocessing must also be recomputed, which takes approximately 95 seconds. The `databases.toml` file is updated to declare the new database, and clients discover it at their next `GET_DB_CATALOG` call.

Dust filtering. UTXOs of 576 satoshis or less, and addresses with more than 100 UTXOs, are filtered during database generation. This removes dust outputs and whale addresses (who should run full nodes), reducing the database size and improving PIR efficiency for typical light-client queries. See Section 2.4 for the rationale.

Acknowledgments

We thank Mingxun Zhou for advice and suggestions on stateful PIR and their implementations. We thank Ling Ren and Pranav Shriram Arunachalam for advice and suggestions on state-of-the-art PIR schemes in various settings, batch PIR, and understanding of the SimplePIR family of protocols.

References

- [ACLS18] Sebastian Angel, Hao Chen, Kim Laine, and Srinath Setty. Pir with compressed queries and amortized query processing. In *IEEE S&P 2018*, 2018.
- [Adv20] Advanced Micro Devices. AMD SEV-SNP: Strengthening VM isolation with integrity protection and more. <https://www.amd.com/en/developer/sev.html>, 2020.
- [AOP25] Bar Alon, Valery Osheter, and Benny Pinkas. Introducing the ALF family: AES-NI-based length- and format-preserving encryption. IACR ePrint 2025/2148, 2025.
- [AR26a] Pranav Shriram Arunachalam and Ling Ren. Efficient single-server stateful PIR using format-preserving encryption. IACR ePrint 2026/437, 2026.
- [AR26b] Pranav Shriram Arunachalam and Ling Ren. Single-server stateful PIR with verifiability and balanced efficiency. In *IEEE S&P 2026*, 2026.
- [BGI15] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing. In *EUROCRYPT 2015*, 2015.
- [BGI16] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing: Improvements and extensions. In *ACM CCS 2016*, 2016.
- [Cas22] Cashu Protocol. Cashu: Chaumian ecash for Bitcoin. <https://cashu.space/>, 2022.
- [CNS07] Jan Camenisch, Gregory Neven, and Abhi Shelat. Simulatable adaptive oblivious transfer. In *EUROCRYPT 2007*, 2007.
- [CR25] Yue Chen and Ling Ren. OnionPIRv2: Efficient single-server PIR. IACR ePrint 2025/1142, 2025.
- [dL24] Leo de Castro and Keewoo Lee. VeriSimplePIR: Verifiability in SimplePIR at no online cost for honest servers. In *USENIX Security 2024*, 2024.
- [DNRS99] Cynthia Dwork, Moni Naor, Omer Reingold, and Larry J. Stockmeyer. Magic functions. In *IEEE FOCS 1999*, 1999.
- [FK23] Dankrad Feist and Dmitry Khovratovich. Fast amortized KZG proofs. IACR ePrint 2023, 2023.
- [GI14] Niv Gilboa and Yuval Ishai. Distributed point functions and their applications. In *EUROCRYPT 2014*, 2014.
- [HDCGZ23] Alexandra Henzinger, Emma Dauterman, Henry Corrigan-Gibbs, and Nickolai Zeldovich. Private web search with tiptoe. In *ACM SOSP 2023*, 2023.
- [HHCG⁺23] Alexandra Henzinger, Matthew M. Hong, Henry Corrigan-Gibbs, Sarah Meiklejohn, and Vinod Vaikuntanathan. One server for the price of two: Simple and fast single-server private information retrieval. In *USENIX Security Symposium 2023*, 2023.
- [HMR12] Viet Tung Hoang, Ben Morris, and Phillip Rogaway. An enciphering scheme based on a card shuffle. In *CRYPTO 2012*, 2012.

- [HPY25] Alexander Hoover, Sarvar Patel, Giuseppe Persiano, and Kevin Yeo. Plinko: Single-server PIR with efficient updates via invertible PRFs. In *EUROCRYPT 2025*, 2025.
- [IET25] IETF Privacy Pass Working Group. Anonymous rate-limited credentials (ARC). IETF draft-ietf-privacypass-arc-crypto-01, 2025.
- [IKOS04] Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. Batch codes and their applications. In *ACM STOC 2004*, 2004.
- [KCG21] Dmitry Kogan and Henry Corrigan-Gibbs. Private blocklist lookups with Checklist. In *USENIX Security 2021*, 2021.
- [Kil88] Joe Kilian. Founding cryptography on oblivious transfer. In *ACM STOC 1988*, 1988.
- [KS06] Mehmet S. Kiraz and Berry Schoenmakers. A protocol issue for the malicious case of Yao’s garbled circuit construction. In *Proceedings of the 27th Symposium on Information Theory in the Benelux*, 2006.
- [KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In *ASIACRYPT 2010*, 2010.
- [LHCG25] Ryan Lehmkuhl, Alexandra Henzinger, and Henry Corrigan-Gibbs. Distributional private information retrieval. In *USENIX Security 2025*, 2025.
- [LMRS24] Baiyu Li, Daniele Micciancio, Mariana Raykova, and Mark Schultz-Wu. Hintless single-server private information retrieval. In *CRYPTO 2024*, 2024.
- [LP11] Yehuda Lindell and Benny Pinkas. Secure two-party computation via cut-and-choose oblivious transfer. In *TCC 2011*, 2011.
- [MCR21] Muhammad Haris Mughees, Hao Chen, and Ling Ren. OnionPIR: Response efficient single-server PIR. In *ACM CCS 2021*, 2021.
- [Mic24] Microsoft Research. Microsoft SEAL: Homomorphic encryption library. <https://github.com/microsoft/SEAL>, 2024.
- [MW22] Samir Jordan Menon and David J. Wu. Spiral: Fast, high-rate single-server PIR via FHE composition. In *IEEE S&P 2022*, 2022.
- [MW24] Samir Jordan Menon and David J. Wu. YPIR: High-throughput single-server PIR with silent preprocessing. In *USENIX Security 2024*, 2024.
- [NP99] Moni Naor and Benny Pinkas. Oblivious transfer with adaptive queries. In *CRYPTO 1999*, 1999.
- [PR01] Rasmus Pagh and Flemming Friche Rodler. Cuckoo hashing. In *Proceedings of the 9th Annual European Symposium on Algorithms (ESA ’01)*, 2001.
- [RLP26] Mayank Rathee, Keewoo Lee, and Raluca Ada Popa. Verifiable PIR with small client storage. In *IEEE S&P 2026*, 2026.
- [RMS24] Ling Ren, Muhammad Haris Mughees, and I Sun. Simple and practical amortized sublinear private information retrieval using dummy subsets. In *ACM CCS 2024*, 2024.
- [Voe11] Thomas Voegtlin. Electrum Bitcoin wallet. <https://electrum.org/>, 2011.
- [W3C19] W3C. WebAssembly specification. <https://webassembly.org/>, 2019.
- [ZPZS24] Mingxun Zhou, Andrew Park, Wenting Zheng, and Elaine Shi. Piano: Extremely simple, single-server PIR with sublinear server computation. In *IEEE S&P 2024*, 2024.